

Chapter 3 Java Features, Part 2

3.1 Classes

A Java program consists of one or more classes. The term *class* refers to a class of objects.

You should know the following concepts and terms:

class	encapsulation
object	class (static) variables and methods
instance of a class	public static final constants
constructor	method signature
new operator	accessor methods
instance variables	mutator (modifier) methods
private instance variables	

An object that belongs to a particular class is also called an *instance* of that class, and the process of creating an object is called *instantiation*.

A class definition may include *constructors*, instance and/or static variables, and methods. A constructor describes how objects of the class can be created (there can be one or several constructors defined in a Java class, or none). The methods describe what an object of this class can do; the *instance variables* describe the object's attributes — the current state of the object.

Comments

Comments are plain English remarks used to explain and document the program's features to the programmer and others who have access to the code. Comments can also be used to temporarily disable ("comment out") sections of code during development and testing.

The `//` characters indicate that the text that follows them, until the end of the line, is a comment. Also, any text between `/*` and `*/` is a comment.

 **The compiler ignores all comments.**

Do not include any comments in your solutions to the exam's free-response questions.

So-called *javadoc* comments are placed before the class header and before each constructor and method. They describe the functionality of the class, the purposes of its instance variables, and constructor and method parameters. Javadoc comments are marked by an extra `*` after the opening comment mark, as in `/** ... */`. It is customary to place an `*` at the beginning of each line in a javadoc comment. Javadoc comments can be used by the *Javadoc* utility tool in the JDK (Java Development Kit) to automatically generate class documentation in HTML format, viewable in a browser.

Javadoc comments may include *preconditions* and *postconditions*. A precondition describes properties of the parameter(s) of a constructor or a method that are assumed to be true. A programmer does not have to validate preconditions.

Do not write code to validate preconditions, as their properties are guaranteed. Although you will not be penalized if your precondition testing code is correct, you will waste valuable time and you will lose points if your validation code is buggy.

A postcondition may describe properties of the return value or a state of an object when the method is exited. For example, a postcondition may specify that the state of the parameter object remains unchanged. You have to make sure that postconditions are satisfied.

~~~~~ Constructors ~~~~~

Constructors describe ways to create an object of a class and initialize the object's instance variables.

All constructors have the same name as the class. Constructors do not have any return data type, not even `void`.

A constructor may take arguments that help define a new object. In particular, it may initialize some of the instance variables in the newly constructed object. If an instance variable is not initialized, it gets a default value: 0 for `int`, 0.0 for `double`, `false` for `boolean`, `null` for a reference type.

A constructor that takes no arguments is called a “no-args” constructor.

When no constructors are included in the class definition, Java provides an implicit no-args constructor, and the instance variables are set to default values according to their data type. This constructor is called the *default constructor*.

The term *argument* (as with a *function argument* in math) refers to the actual values passed to a constructor or method.

This is evident in such usage as “no-args” constructor, `IllegalArgumentException`, and `main(String[] args)`.

Suppose you have defined a class `School`:

```
public class School
{
    // constructor
    public School(String name, int numStudents) { < code not shown > }
    ...
}
```

Then you can create a `School` object in a *client class* (a class that uses `School` objects), declare a variable of the type `School`, and set it equal to a reference to the newly created object. A new object is created using the `new` operator. For example:

```
School sch1 = new School("Gifted and Talented Magnet", 1200);
```

If the `School` class has instance variables `name` and `numStudents` —

```
public class School
{
    ...
    private String name;
    private int numStudents;
    ...
}
```

— the constructor can set them to the values of the arguments passed to it:

```
public School(String nm, int num)
{
    name = nm;
    numStudents = num;
}
```

Objects that are no longer accessible in the program — that is, no longer referred to by any variable — are automatically destroyed and the memory they occupy is recycled.

This mechanism is called *garbage collection*.

~~~~~ public *and* private ~~~~~

Instance and static variables and methods may be *public* or *private*.

Private variables and methods can only be referred to within the source code of the class they are defined in.

Public variables and methods are accessible anywhere in the program.

The concept of “privacy” applies to the source code of the class as a whole and not to individual objects. Different objects of the same class have full access to each other’s private variables and methods, and can even modify the values of each other’s private variables.

In the AP Java subset, constructors are always public.

It is common practice (and a requirement on the AP CSA exam) to make all instance variables private.

Private variables and methods hide the implementation details of a class from other classes, its clients. This concept is known as *encapsulation*. A *client* class uses your class by invoking constructors and calling public methods of your class. In general, it is a good idea to supply as little information to client classes as possible. For example, if a method is used only internally within a class, it should be made private.

~~~~~ Accessors and mutators ~~~~~

When all instance variables are private, they are not directly accessible in client classes. It is common to provide special public methods, called *accessors*, that return the values of instance variables. For example:

```
public class School
{
    private String name;
    ...
    public String getName()
    {
        return name;
    }
    ...
}
```

Accessors' names often start with "get"; accessors can have any name, but starting with "get" makes their purpose easier to remember. Accessors do not change the state of the object.

A public method that sets an instance variable to a new value is called a *mutator* (or *modifier*). Mutators' names often start with "set"; for example:

```
public class School
{
    private String name;
    ...
    public void setName(String nm)
    {
        name = nm;
    }
    ...
}
```

The "accessor" and "mutator" designations are somewhat informal — a class may have a method that sets an instance variable to a new value and at the same time returns the old value.

_____ this _____

The Java keyword `this` refers to the object whose constructor or method is running. You can pass `this` as an argument to a method. For example, suppose a class `Student` is defined and it has a method that registers this student for a course, represented by an `ArrayList<Student>`. Something like this:

```
public class Student
{
    ...
    public void register(ArrayList<Student> course)
    {
        course.add(this);
    }
}
```

```

public class School
{
    public static final int HIGHEST_GRADE = 12;
    private static int numSchools;
    private static int[] totalNationalEnrollmentByGrade;
    ...
    public static void initializeStatistics()
    {
        numSchools = 0;
        totalNationalEnrollmentByGrade = new int[HIGHEST_GRADE + 1];
        // +1 for kindergarten
    }
    ...
}

```

Static variables can be initialized in a class constructor, and they can be accessed and modified in instance methods. For example:

```

public class School
{
    // Static variables:
    public static final int HIGHEST_GRADE = 12;
    private static int numSchools;
    private static int[] totalNationalEnrollmentByGrade;

    // Static methods:
    public static int getNationalEnrollment(int grade)
    {
        return totalNationalEnrollmentByGrade[grade];
    }
    ...

    // Instance variables:
    private int[] enrollmentByGrade;
    ...

    // Constructor:
    public School(int[] numStudents)
    {
        numSchools++;

        enrollmentByGrade = new int[HIGHEST_GRADE + 1];

        for (int grade = 0; grade <= HIGHEST_GRADE; grade++)
        {
            enrollmentByGrade[grade] = numStudents[grade];
            totalNationalEnrollmentByGrade[grade] += numStudents[grade];
        }
        ...
    }

    // Instance methods:
    public void enrollOneStudent(int grade)
    {
        enrollmentByGrade[grade]++;
        totalNationalEnrollmentByGrade[grade]++;
    }
    ...
}

```

Static methods cannot access or modify any instance variables and cannot refer to `this`, because `this` is undefined when a static method is running.

20

Consider a class with the following variables.

```
public class TestPow2
{
    private static int[] powersOfTwo = {1, 2, 4, 8, 16};
    private int num;
    ...
}
```

Which of the following methods inside the `TestPow2` class will compile with no errors?

- I.

```
public int pow2()
{
    return powersOfTwo[num];
}
```
- II.

```
public static int pow2(int exp)
{
    return powersOfTwo[exp];
}
```
- III.

```
public static int pow2()
{
    return powersOfTwo[num];
}
```

- (A) I only
- (B) II only
- (C) I and II only
- (D) II and III only

 The code in Option I would compile, because here the instance (non-static) method `pow2` can work with both the instance variable `num` and the static variable `powersOfTwo`. The code in Option II would compile, too, because here the static method `pow2` works only with its own parameter and with the static variable `powersOfTwo`. The code in Option III would cause a syntax error, because the static method `pow2` attempts to access the instance variable `num`. The answer is C. 

All methods of the Java library class `Math` are static methods, including `abs`, `sqrt`, `pow`, and `random`.

For your convenience, the `Math` class also includes the public static final “variables” `Math.PI` (which represents π , the ratio of a circle’s circumference to its diameter) and `Math.E` (which represents e , the base of the natural logarithm). `Math` also has the static methods `min` and `max`, which return the smaller and the larger, respectively, of two numbers passed to the method as parameters (ints or doubles); you don’t need to know this for the exam, but it might be useful.

Outside the class, public static methods are called and public static constants are accessed using the dot notation, with the class’s name as the prefix. For example:

```
double volume = 4.0 / 3.0 * Math.PI * Math.pow(r, 3);
```

3.3 Calling Methods

In Java, every method belongs inside a class.

It is universal Java style that all method names start with a lowercase letter.

An *instance* method is called for a particular object; the object’s name and a dot are used as a prefix in the call, as in `obj.someMethod(...)`. If an object’s method calls another method of the same object, the prefix is not needed and you write simply `otherMethod(...)`, which is the same as `this.otherMethod(...)`.

Static methods belong to the class as a whole and are called using a prefix consisting of the class’s name with a dot. For example: `Math.max(x, y)` or `School.getNationalEnrollment(...)`.

A method takes a specific number of parameters of specific data types, listed in a particular order.

A method can be defined with a variable number of parameters, as in `System.out.printf`; however, this feature is not in the AP Java subset.

Some methods take no parameters, such as `ArrayList’s size()` or `String’s length()`. A method call may include an expression as an argument; in that case, the expression is evaluated first and the result is passed to the method. An expression may include calls to other methods. For example:

```
double x, y;
...
x = Math.sqrt(Math.abs(2*y - 1));
```

A method usually returns a value of the specified data type, but a `void` method does

not return any value. The return type is specified in the method's header. The return value is specified in the return statement.

It is an error to read the new values for a method's parameters from `System.in` inside the method. It is also an error to print the return value to `System.out` inside a method (when it is not requested), and another error to omit a return statement in a non-void method.

For example:

```
// Returns the sum of all integers from 1 to n.
// Precondition: n >= 1
public int addNumbers(int n)
{
    int sum = 0;

    n = System.in.read();

    for (int k = 1; k <= n; k++)
    {
        sum += k;
    }

    System.out.println(sum);

    return sum;
}
```

Error: n is passed to this method from main or from another calling method

Error: Not specified in the method description

21

`Math`'s static method `min` returns the value of the smaller of two integers. If `a`, `b`, `c`, and `m` are integer variables, which of the following best describes the behavior of a program that contains the following statement?

```
m = Math.min(Math.min(a, c), Math.min(b, c));
```

- (A) The statement has a syntax error and will not compile.
- (B) The program will run but will go into an infinite loop.
- (C) `a` will get the smaller value of `a` and `c`; `b` will get the smaller value of `b` and `c`; `m` will get the smallest value of `a`, `b`, and `c`.
- (D) `m` will be assigned the smallest of the values `a`, `b`, and `c`.

23

Consider the following method.

```
public void accumulate(int[] a, int n)
{
    while (n < a.length)
    {
        a[n] += a[n-1];
        n++;
    }
}
```

What is the output from the following code?

```
int[] a = {1, 2, 3, 4, 5};
int n = 1;
accumulate(a, n);
for (int k = 0; k < a.length; k++)
    System.out.print(a[k] + " ");
System.out.println(" " + n + " ");
```

- (A) 1 2 3 4 5 *1*
- (B) 1 2 3 4 5 *5*
- (C) 1 3 5 7 9 *5*
- (D) 1 3 6 10 15 *1*

 n is passed to `accumulate` by value, so `accumulate` cannot change n . This rules out Choices B and C. (Inside `accumulate`, n acts like a local variable.) a is passed to `accumulate` as a reference, so `accumulate` can change its values. Starting at $n = 1$, it adds the value of the previous element to the current one. As the name of the method implies, this sets `a[k]` to the sum of all the elements up to and including `a[k]` in the original array. The answer is D. 

_____ return _____

A method that is not void must return a value of the designated type using the `return` statement. `return` works with any expression, not just variables. For example:

```
return (-b + Math.sqrt(b*b - 4*a*c)) / (2*a);
```

An often-overlooked fact is that a boolean method can return the value of a Boolean expression. For example, you can write simply

```
return x >= a && x <= b;
```

as opposed to the redundant and verbose

```
if (x >= a && x <= b)
    return true;
else
    return false;
```

A void method can use a return (within if or else) to quit early, but there is no need for a return at the end of the method. It is okay, and often advisable, to have multiple return statements in a method. For example, the following *recursive* method is well-written:

```
// Returns the index of target in a sorted array
// among a[i], ..., a[j] or -1 if not found
public static int binarySearch(int[] a, int i, int j, int target)
{
    if (i > j)
        return -1;

    int m = (i + j) / 2;
    if (target == a[m])
        return m;

    if (target < a[m])
        return binarySearch(a, i, m-1, target);
    else
        return binarySearch(a, m+1, j, target);
}
```

~~~~~ Returning objects ~~~~~

A method's return type can be a class, and a method can return an object of that class. Often a new object is created in the method and then returned from the method. For example:

```
public String getFullName(String firstName, String lastName)
{
    return firstName + " " + lastName;
}
```

Or:

```
public Location adjacentSouth(Location loc)
{
    int r = loc.getRow();
    int c = loc.getCol();
    return new Location(r+1, c);
}
```

Or simply:

```
public Location adjacentSouth(Location loc)
{
    return new Location(loc.getRow()+1, loc.getCol());
}
```

A method whose return type is a class can also return a `null` (a reference with a zero value, which indicates that it does not refer to any valid object). For example:

```
/**
 * Precondition: listOfNames and listOfAddresses are instance variables
 * of this class and they hold valid data
 */
public String getAddress(String name)
{
    for (int i = 0; i < listOfNames.length; i++)
    {
        if (listOfNames[i].equals(name))
            return listOfAddresses[i];
    }
    return null; // not found
}
```

If a method returns an `ArrayList`, write the full `ArrayList` type, including its elements' type in angle brackets, as the method's return type. For example:

```
public ArrayList<Integer> getCourseNumbers()
{
    ArrayList<Integer> courseNumbers = new ArrayList<Integer>();
    ...
    return courseNumbers;
}
```

~~~~~ *Overloaded methods* ~~~~~

The method's name together with the ordered list of the types of its parameters is called the method's *signature*.

The signature of a method that takes no parameters is its name and empty parentheses — (). The method's return type is not part of its signature.

Methods of a class that have the same name but a different number and/or different types of parameters are called *overloaded* methods. The order of parameters, if they are of different types, also matters.

The compiler treats overloaded methods as different methods. It figures out which one to call depending on the number, the types, and the order of the arguments passed to the method.

The `String` class, for example, has two overloaded `substring` methods:

```
String substring(int from)
String substring(int from, int to)
```

If you call `"Happy".substring(2)` then the first method will be called, but if you call `"Happy".substring(1, 4)` then the second method will be called. Another example of overloading is `Math.abs(x)`, which has different versions of the static method `abs`, including `abs(int)` and `abs(double)`. `System.out.print(x)` has overloaded versions for all primitive data types as well as for `String` and `Object`.

The `ArrayList` class has two overloaded `add` methods: `add(obj)`, which adds `obj` at the end of the list, and `add(index, obj)`, which inserts `obj` at position `index`.

Overloading methods is basically a stylistic device. You could instead give different names to different forms of a method, but then it would be harder to remember their names. Overloaded methods don't need to have the same return type, but they often do, because they perform similar tasks, such as in `String`'s `substring` methods. A different return type alone is not enough to overload a method.

All constructors of a class have the same name, so they are overloaded by definition and must differ from each other in the number and/or types of their parameters.

24

Consider the following incomplete class declaration.

```
public class Date
{
    public Date()
    { /* implementation not shown */ }

    public Date(String monthName, int day, int year)
    { /* implementation not shown */ }

    public void setDate(int month, int day, int year)
    { /* implementation not shown */ }

    /* instance variables and other constructors and
       methods not shown */
}
```

We wish to modify the `Date` class to make it possible to initialize variables of the type `Date` with month (given as a month name or number), day, and year information when they are declared, as well as to set their values later using the method `setDate`. For example, the following code segment should define and initialize three `Date` variables:

```
Date d1 = new Date();
d1.setDate("May", 11, 2026);
Date d2 = new Date("June", 30, 2030);
Date d3 = new Date(6, 30, 2030);
```

Which of the following best describes the additional features that should be present?

- (A) A constructor with three `int` parameters
- (B) An overloaded version of `setDate` with one `String` and two `int` parameters
- (C) Both a constructor with three `int` parameters and an overloaded version of `setDate` with three `int` parameters
- (D) Both a constructor with three `int` parameters and an overloaded version of `setDate` with one `String` and two `int` parameters

 This is a wordy but simple question. Just match the declarations against the provided class features:

<code>Date d1 = new Date();</code>	_____	✓ <code>Date()</code>
<code>Date d2 = new Date("June", 30, 2010);</code>	_____	✓ <code>Date(String, int, int)</code>
<code>Date d3 = new Date(6, 30, 2010);</code>	_____	<code>Date(int, int, int)</code>
<code>d1.setDate("May", 11, 2004);</code>	_____	<code>void setDate(String, int, int)</code>
<i>not used</i>	_____	✓ <code>void setDate(int, int, int)</code>

As we can see, what's missing in the class definition is a constructor with three `int` parameters and a version of `setDate` with one `String` and two `int` parameters. The answer is D. 

Three review questions

Questions 25-27 refer to the following partial class definition:

```
public class TicketSales
{
    /** Holds the name of the movie */
    private String name;

    /** sales[0], ..., sales[51] hold sales totals for 52 weeks */
    private double[] sales;

    public TicketSales(String movieName)
    { /* implementation not shown */ }

    /**
     * Sets box office receipts for a given week.
     * Precondition: 1 <= week <= 52
     */
    public void setWeekSales(int week, double dollars)
    { /* implementation not shown */ }

    /**
     * Finds and returns the week with best sales.
     */
    private int findBestWeek() { < implementation not shown > }

    /* Other methods not shown */
}
```

25

The method `findBestWeek` is declared private because

- (A) `findBestWeek` is not intended to be used by clients of the class.
- (B) `findBestWeek` is intended to be used only by clients of the class.
- (C) Methods that work with private instance variables of the array type cannot be public.
- (D) Methods that return a value cannot be public.

 In this question, only the first two choices deserve any consideration — the other two are fillers. You might get confused for a moment about what “client” means, but common sense helps: a client is anyone who is not yourself, so if a client needs to use something of yours, you have to make it public. Private things are for your class, not for clients. The answer is A. 

26

The constructor for the `TicketSales` class initializes the `sales` array to hold 52 values. Which of the following statements will do that?

- (A) `double sales[52];`
- (B) `double sales = new double[52];`
- (C) `double[] sales = new double[52];`
- (D) `sales = new double[52];`

 This is a syntax question. Choice A has invalid syntax. Choice B assigns an array to a `double` variable — a syntax error. Both Choices C and D appear syntactically plausible and in fact either one will compile with no errors. But Choice C, instead of initializing an instance variable `sales`, will declare and initialize a local variable with the same name. This is a common nasty bug in Java programs. The answer is D. 

27

Given the declaration

```
TicketSales movie = new TicketSales("Monsters, Inc.");
```

which of the following statements sets the third-week sales for that movie to 245,000?

- (A) `movie = TicketSales(3, 245000.00);`
- (B) `setWeekSales(movie, 3, 245000.00);`
- (C) `movie.setWeekSales(3, 245000.00);`
- (D) `setWeekSales(3, 245000.00);`

 This is another syntax question. The variable `movie` of the type `TicketSales` is defined outside the class, in a client of the class. The key word in this question is “sets.” It indicates that a method — a modifier — is called. The way to call a method from a client class is with dot notation. (Besides, Choice A assumes that there is a constructor with two parameters; Choice B looks like a call to non-existing methods; Choice D forgets to mention `movie` altogether.) The answer is C. 

3.4 Random Numbers

Random numbers simulate chance in computer programs. For example, if you want to simulate a roll of a 6-sided die, you need to obtain a random number from 1 to 6 (with any one of these values appearing with the same probability). “Random” numbers are not truly random — their sequence is generated using a certain formula — but they are good enough for many applications.

One way to get random numbers in a Java program is to call the static method `random` of the `Math` class. It returns a random double from 0 (inclusive) to 1 (exclusive). To get a random integer from 1 to n use:

```
int r = (int)(n * Math.random()) + 1;
```

28

Which of the following is a list of all possible outputs of the following code segment?

```
String memo = "MEMO";
System.out.print "[" + memo.substring((int)(3 * Math.random()),
                                     (int)(3 * Math.random()) + 2) + "];
```

- (A) [ME], [EM]
- (B) [ME], [EM], [MO]
- (C) [], [M], [ME], [EM], [MO]
- (D) [], [M], [E], [ME], [EM], [MO], [MEM], [EMO], [MEMO]

 The two calls to `Math.random()` look the same, but they return different values — two successive values in the random number sequence. The “from” parameter of `memo.substring` can be 0, 1, or 2, and the “to” parameter can be 2, 3, or 4. Any combination of these from and to values results in a valid substring (including `substring(2, 2)`, which returns an empty string). The answer is D. 

 Another way to get random numbers relies on the `java.util.Random` class, but that class is not in the AP Java subset and should be avoided.

3.5 Class Libraries and the API

The JDK (Java Development Kit) includes an extensive library of classes, which are arranged into packages. A small selection of these classes and a subset of their methods may be tested on the AP CSA exam. These classes and methods are listed in the *Java Quick Reference*, which is provided for you during the exam in both the multiple-choice and free-response sections.

The complete documentation of the Java library is available in the API specification. *API* stands for *Application Programming Interface*.*

The Java library API documentation is generated from the code using the javadoc tool. It can be found on Oracle's Java website; for example, the API documentation of Java Version 23 is at <https://docs.oracle.com/en/java/javase/23/docs/api>.

You normally have to import library classes into your code using the `import` statement. For example:

```
import java.util.Scanner;  
import java.io.File;
```

However, you can avoid importing a library class if you use its *fully-qualified name* — which includes the package name — every time you reference it.

Classes from the `java.lang` package, such as `String`, `Integer`, `Math`, and `System`, are imported automatically and do not require import statements.

When answering the exam's free-response questions, assume that all import statements are present and never supply your own.

In large projects or organizations, programmers can create their own library of reusable classes with its own API.

* The March 2025 release of the CED has typos on pages 17, 26, and 39, incorrectly calling it “Application *Program* Interface”; on page 39, the typo is only in the topic title, as the “Essential Knowledge” paragraph correctly uses “*application programming interface*.”

3.6 Program Design and Development Methodology

Computer science courses emphasize problem solving, as opposed to just programming in a particular language or using specific hardware platforms. The exam topics related to general software design and development methodology emphasize *procedural abstraction*, *data abstraction*, and *functional decomposition* (*method decomposition*). Here is a very brief glossary of these and other relevant terms:

Procedural abstraction is specifying and using procedures and functions (methods) without knowing the details of their implementation.

Data abstraction is specifying and using data structures without knowing the details of their implementation.

Functional decomposition (also known as *method decomposition*) is breaking down large methods into several smaller methods.

A *data structure* (also known as *data collection*) is a way of organizing data, combined with ways of accessing and manipulating the data. For example, a two-dimensional array, together with methods or operators for accessing and modifying the values of its elements, is a data structure that may be useful for representing tables, grids, matrices, or images.

Specifications are a detailed description of what a piece of software should accomplish and how it should behave and interact with the user. Specifications may be given for a whole system, one module, or even one class or method.

Encapsulation and *information hiding* is the practice of making private all instance variables and helper methods that are used only inside the class. The clients of such a class can use the class only through its public constructors and public methods.

Artificial intelligence (AI) is the simulation of intelligent behavior by a computer system or algorithm. The use of artificial intelligence when developing code may decrease the amount of time it takes to develop code. The use of artificial intelligence in developing code has the potential to create programs that contain various types of bias, including bias toward different groups of people.

These are very general concepts, and it is not easy to come up with multiple-choice or free-response questions that test in-depth understanding of these concepts. In past exams, design and implementation questions have been limited to specific data structures and algorithms, which sometimes used some of these terms in their descriptions. In particular, AI concepts won't be tested on the exam.

3.7 The “Class” Question

Free-response Question 2 of the exam is a “class” question that asks you to design and write a short class, including instance variables, one constructor, and one method.

 **The “class” question will consist of only one part: writing the class.**

When designing and coding your class, pay special attention to the following:

- Use reasonable and consistent style, with proper indentation and generous spacing between statements.
- Do not include comments — they are not required and will waste your time.
- Use the provided name for your class, and use the provided name for the method if given. Use meaningful names for the constructor and method parameters, and for local and instance variables. A class name starts with an uppercase letter; the names of all methods, their parameters, and instance variables start with lowercase letters. Make sure that the constructor has the same name as the class and has no return type, not even void.
- Make all instance variables private and group them together at the top of the class.
- Make the method public.
- Specify an appropriate return type for the method.

3.8 Inheritance

As we have stated in the preface, the College Board has eliminated all object-oriented programming (OOP) topics from the exam. Inheritance, subclasses and superclasses, class hierarchies, and polymorphism are all excluded. However, the CED includes two “Essential Knowledge ” items related to inheritance and an exclusion statement. (Figure 3-1).

1.12.A.2

A class hierarchy can be developed by putting common attributes and behaviors of related classes into a single class called a *superclass*. Classes that extend a superclass, called *subclasses*, can draw upon the existing attributes and behaviors of the superclass without replacing these in the code. This creates an *inheritance relationship* from the subclasses to the superclass.

✖ EXCLUSION STATEMENT—*Designing and implementing inheritance relationships are outside the scope of the AP Computer Science A course and exam.*

1.12.A.3

All classes in Java are subclasses of the `Object` class.

Figure 3-1. “Essential Knowledge” CED items related to inheritance

Inheritance defines the *is-a* relationship between classes: *P* is-a *Q* if the class *P* is a subclass of *Q*. This is not to be confused with the *has-a* relationship. Class *Q* *has-a* *P* if the definition of *Q* includes an instance variable of the type *P*.

It is useful to understand that any Java class is a subclass of the class `Object`. So any object *is-a(n)* `Object`. Any class inherits all public methods of `Object`. Of particular interest to us are two of them:

```
boolean equals(Object other)
```

and

```
String toString()
```

These two methods, as implemented in `Object`, are not very useful, because `equals` compares the address of this object to the address of another object and `toString` returns the address of this object in hexadecimal notation.

The role of these methods is only to declare their presence and to let any class override them in a meaningful way. If `toString` is defined in a class, it is called when an object of that class is concatenated with a string or when an object of the class is printed by `System.out.print` or `System.out.println`. The `equals` method is called, for example, from the `ArrayList`'s `contains` method. The `Integer`, `Double`, `String`, `ArrayList`, and many other Java library classes override `Object`'s `equals` and `toString`.

Chapter 4 Algorithms

4.1 Iterations

Most programming languages provide iteration control structures, such as the `while` and `for` loops in Java. Simple loops are good for iterating (repeating the same operation) over a range of numbers or over the elements of a one-dimensional array or a list.

A `for` loop is a convenient and idiomatic way to *traverse* a one-dimensional array:

```
for (int k = 0; k < a.length; k++)
{
    System.out.println(a[k]);    // ... or do whatever you need to do
                                // with each element
}
```

or

```
for (T x : a)
{
    System.out.println(x);
}
```

where T is the data type of the elements of `a`.

Working with two-dimensional arrays usually requires *nested* loops. The following code, for example, traverses a two-dimensional array `m`:

```
int nRows = m.length, nCols = m[0].length;

for (int r = 0; r < nRows; r++)
{
    for (int c = 0; c < nCols; c++)
    {
        System.out.println(m[r][c]);    // ... or do whatever...
    }
}
```

Notice that braces are optional if the body of the loop has only one statement:

```
for (int i = 1; i < n; i++)
    for (int j = 0; j < i; j++)
        if (a[i] == a[j])
            count++;
```

In “triangular” nested loops, the outer loop may run, say, for i from 1 to $n-1$, and the inner loop may run for j from 0 to $i-1$. In the above example the inner loop runs i times for $i = 1, \dots, n-1$, so the total number of comparisons is

$$1 + 2 + \dots + (n-1) = \frac{n(n-1)}{2}$$

The for-each loop (also known as “enhanced for loop”) makes it easy to traverse an array or ArrayList:

```
ArrayList<String> list = new ArrayList<String>();
...
for (String s : list)
{
    ...
}
```

The Course and Exam Description (CED) lists the following standard algorithms that use an array or an ArrayList:

- determine a minimum or maximum value
- compute a sum or average
- determine if at least one element or all elements have a particular property
- determine the number of elements having a particular property
- access all consecutive pairs of elements
- determine the presence or absence of duplicate elements
- shift or rotate elements left or right or reverse the order of the elements

It’s a good idea to practice with these before the exam. In addition, practice with ArrayList algorithms that require inserting and deleting elements.

~~~~~ *Max and min* ~~~~~

A common example of using loops is to find a maximum or minimum value (or its position) in an array or in an ArrayList. For example:

```

/**
 * Returns maxValue such that maxValue >= a[k] for any 0 <= k <= n-1
 * and maxValue = a[k] for some k.
 * Preconditions: n >= 1; array a holds values a[0], ..., a[n-1]
 */
public double max(double[] a, int n)
{
    double maxValue = a[0];

    for (int k = 1; k < n; k++)
    {
        if (a[k] > maxValue)
            maxValue = a[k];
    }
    return maxValue;
}

```

29

Consider the following method.

```

public int mysteryMax(int[] a)
{
    int m = 0;

    for (int i = 0; i < a.length; i++)
    {
        int sum = 0;

        for (int k = i; k < a.length; k++)
        {
            sum += a[k];
            if (sum > m)
                m = sum;
        }
    }

    return m;
}

```

If a contains -1, -3, 2, -3, 2, 1, what value will be returned by `mysteryMax(a)`?

- (A) -2
- (B) -1
- (C) 2
- (D) 3

 The method returns the largest sum of any number of consecutive elements in a (or 0 if all the sums are negative). The answer is D. 

30

Consider the following method.

```
/**
 * Returns the largest sum of any two elements.
 * Preconditions: n >= 2; a[0] ... a[n-1] are filled with values
 */
public double maxSum(double[] a, int n)
{
    < missing code >
}
```

Which of the following code segments can replace *< missing code >* so that the method works as specified?

- I. `double max = a[0] + a[1];`
 `for (int i = 1; i < n; i++)`
 `for (int j = 0; j < i; j++)`
 `if (a[i] + a[j] > max)`
 `max = a[i] + a[j];`
 `return max;`
- II. `double max1 = a[0], max2 = a[0];`
 `for (int i = 1; i < n; i++)`
 `if (a[i] > max1)`
 `max1 = a[i];`
 `for (int i = 1; i < n; i++)`
 `if (a[i] != max1 && a[i] > max2)`
 `max2 = a[i];`
 `return max1 + max2;`
- III. `double max1 = a[0], max2 = a[1];`
 `if (a[1] > a[0])`
 {
 `max1 = a[1];`
 `max2 = a[0];`
 }
 `for (int i = 2; i < n; i++)`
 {
 `if (a[i] > max1)`
 {
 `max2 = max1;`
 `max1 = a[i];`
 }
 `else if (a[i] > max2)`
 `max2 = a[i];`
 }
 `return max1 + max2;`

- (A) I only
- (B) I and II only
- (C) I and III only
- (D) I, II, and III

🔗 This is a lot of code for one question, so we need to focus on the key points. The code in Option I is inefficient but most straightforward: using triangular nested loops, it generates sums for all the different pairs of elements and chooses the largest of them. Option II is based on a different idea: finding the largest value, and then the second largest value, in two separate traversals of the array. But it has two problems. First, if the largest value happens to be `a[0]`, then the second `for` loop will never update `max2`. Second, it will fail if the largest value appears in the array more than once. The method description states that the method is looking for the largest sum of any two elements, even if they have the same value.

To work, Option II would need a couple of minor fixes:

```
int iMax1 = 0, iMax2 = 0;

for (int i = 1; i < n; i++)
    if (a[i] > a[iMax1])
        iMax1 = i;

if (iMax1 == 0)
    iMax2 = 1;

for (int i = 1; i < n; i++)
    if (i != iMax1 && a[i] > a[iMax2])
        iMax2 = i;

return a[iMax1] + a[iMax2];
```

In Option III, we find both the largest and the second-largest elements in one sweep (and they can have the same value). Notice how the largest element becomes the second largest when we find another one with a greater value. It works fine. The answer is C. 🔗

Insert in order

Many applications, including Insertion Sort, require you to insert a value into a sorted array while preserving the order:

```
/**
 * Shifts values a[k], ..., a[n-1] appropriately into
 * a[k+1], ..., a[n] and inserts newValue into a[k] so that the
 * ascending order is preserved.
 * Preconditions: a[0] <= a[1] <= ... <= a[n-1];
 *                  n < a.length
 */
public void insertInOrder(int[] a, int n, int newValue)
{
    // Shift values to the right by one until you find the
    // place to insert:

    int k = n;          // Start at the end

    while (k > 0 && a[k-1] > newValue)
    {
        a[k] = a[k-1];
        k--;
    }
    a[k] = newValue;
}
```

In the above code, we shift values in the array to the right by one to create a vacant slot and then insert the new value into the vacancy thus created. Notice that the shifting has to proceed from the end, so that each shifted value is placed into a vacant slot and does not overwrite any data (Figure 4-1).

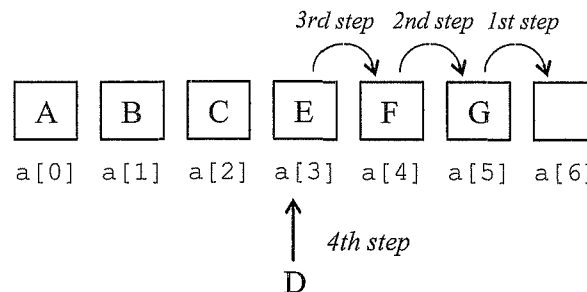


Figure 4-1. Inserting a new value in the middle of a sorted array

31

Consider an array `a` that contains `n` integer values sorted in ascending order (`n < a.length`). Which of the following code segments correctly inserts `newValue` into `a`, preserving the ascending order?

I.

```
for (int k = n; k > 0; k--)
{
    if (a[k-1] <= newValue)
    {
        a[k] = newValue;
        k = 0;
    }
    else
        a[k] = a[k-1];
}
```

II.

```
int k = n;
while (k > 0 && a[k-1] > newValue)
{
    a[k] = a[k-1];
    k--;
}
a[k] = newValue;
```

III.

```
int k = 0;

while (k < n && a[k] < newValue)
    k++;

for (int j = n-1; j >= k; j--)
{
    a[j+1] = a[j];
}
a[k] = newValue;
```

- (A) I only
- (B) II only
- (C) I and II only
- (D) II and III only

 When you have to decide whether such code is correct, check the boundary conditions first: does it work if you have to insert the value at the very beginning or at the very end of the array? The code in Option I, for example, looks good at first — similar to the `insertInOrder` method described above. But if `newValue` is smaller than all the values in the array, nothing is inserted. Option II is equivalent to the `insertInOrder` code above. Option III uses a more step-wise approach: first find the place to insert `newValue`, then shift the values from that place towards the end, then insert. In Option III, it is sufficient to check that this code works when `newValue` is the smallest or largest value — that means there are no tricks. The answer is D. 

4.2 Sequential Search and Binary Search

A typical application of a simple loop is *Sequential Search*:

```
/**
 * Returns the first value of pos, such that
 * 0 <= pos < n and a[pos] == target;
 * returns -1 if target is not among a[0], ..., a[n-1].
 * Precondition: a.length >= n
 */
public int sequentialSearch(int[] a, int n, int target)
{
    for (int k = 0; k < n; k++)
    {
        if (a[k] == target)
            return k;
    }
    return -1;
}
```

Sequential Search works for any array; the values in the array may be in random order. If an array is sorted — that is, if its elements are arranged in ascending or descending order — then Binary Search is a much more efficient searching algorithm.

Binary Search is a “divide and conquer” algorithm for quickly finding a target value in a sorted array. Suppose the array is sorted in ascending order. We take an element in the middle (or approximately in the middle) of the remaining search range (which is the whole array at the start) and compare that element to the target. If they are equal, we are done. If the target is greater, we continue the search in the right half of the remaining search range; if it’s smaller, we continue in the left half.

For example:

```
/**
 * Returns the position of the element equal to target, or -1 if target
 * is not in the array.
 * Precondition: array a contains n values sorted in ascending order
 */
public int binarySearch(int[] a, int n, int target)
{
    int left = 0;
    int right = n - 1;

    while (left <= right)
    {
        int middle = (left + right)/2;
        if (target == a[middle])
            return middle;
        else if (target < a[middle])
            right = middle - 1;    // continue search in the left half
        else
            left = middle + 1;    // continue search in the right half
    }
    return -1;
}
```

Binary Search in an array of $2^k - 1$ elements requires at most k iterations. In other words, Binary Search in an array of n elements requires up to $\log_2 n$ iterations. Thus, in an array of 1,000,000 elements it would need at most 20 iterations. By comparison, Sequential Search in an array of n elements takes, on average, $n/2$ iterations, and in the worst case it takes n iterations.

32

Suppose that two programs, one using Binary Search and the other using Sequential Search, take (on average) the same amount of time to find a random target value in a sorted array of 30 elements. Roughly how much faster than the Sequential Search program will the Binary Search program run on an array of 1000 elements?

- (A) 2 times faster
- (B) 10 times faster
- (C) 16 times faster
- (D) 33 times faster

 Binary Search takes up to 5 iterations for 30 elements ($32 = 2^5$) and 10 iterations for 1000 elements ($1024 = 2^{10}$). So, Binary Search will take roughly twice as long on a 1000-element array as on a 30-element array. Meanwhile, Sequential Search will run roughly 33 times longer on a 1000-element array than on a 30-element array ($1000/30 \approx 33$). This means that on 1000 elements, Binary Search will be $33/2 = 16.5$ times faster than Sequential Search. The answer is C. 

33

An email address is a string made up of alphanumeric characters, one or several “dots,” and one “@” character. The short substring after the last dot is called the top-level domain (TLD). For example, in `jane.lee@math.thebestschool.edu`, “edu” is the TLD. Which of the following algorithms can be used to find the position of the TLD?

- I. A modified Sequential Search in which we scan through the whole array, keeping track of the last occurrence of a given character
 - II. A modified Sequential Search that proceeds backward, starting at the end of the array
 - III. A modified Binary Search in which each alphanumeric character is treated as ‘0’ and each dot and @ is treated as ‘1’
- (A) I only
 (B) II only
 (C) I and II only
 (D) II and III only

 The task is basically to find the last dot in a string. Algorithm I is not the most efficient, but it works:

```
for (int k = 0; k < email.length(); k++)
    if (email.charAt(k) == '.')
        dotPos = k;

// Characters, charAt, and char constants are not
// in the AP Java subset

return dotPos;
```

Algorithm II works a bit faster:

```
for (int k = email.length() - 1; k >= 0; k--)
    if (email.charAt(k) == '.')
        return k;
```

The description of Algorithm III tries to confuse you with the binary number system, which has no relation to Binary Search. The latter won’t work here because the string is not sorted and dots are scattered among alphanumeric characters. The answer is C. 

4.3 Selection Sort and Insertion Sort

Sorting means arranging a list of items in ascending or descending order according to either the value of each item or some key that is part of each item.

Selection Sort and ***Insertion Sort*** are called *quadratic sorts* because they use a nested loop and the number of required comparisons is approximately proportional to n^2 .

Selection Sort

In *Selection Sort* we iterate for k from n down to 2: we find the largest among the first k elements and swap it with the k -th element. Here is one version of Selection Sort:

```
/**
 * Sorts n values in array a in ascending order.
 * Preconditions: array a contains a[0], ..., a[n-1]; n >= 1
 */
public void selectionSort(int[] a, int n)
{
    for (int k = n; k >= 2; k--)
    {
        int maxPos = 0;
        for (int i = 1; i < k; i++)
        {
            if (a[i] > a[maxPos])
                maxPos = i;
        }
        // Swap a[maxPos] and a[k-1]:
        int temp = a[maxPos]; a[maxPos] = a[k-1]; a[k-1] = temp;
    }
}
```

Or we can use a while loop:

```
public void selectionSort(int[] a, int n)
{
    while (n > 1)
    {
        int maxPos = 0;
        for (int i = 1; i < n; i++)
        {
            if (a[i] > a[maxPos])
                maxPos = i;
        }
        int temp = a[maxPos]; a[maxPos] = a[n-1]; a[n-1] = temp;
        n--;
    }
}
```

In another variation of Selection Sort, we find the smallest among the elements $a[k], \dots, a[n-1]$ and swap it with $a[k]$ (for $k = 0, \dots, n-2$).

In Selection Sort, the inner loop runs $k-1$ times, for $k = n, n-1, \dots, 2$.

The total number of comparisons in Selection Sort is always the same:

$$(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$$

~~~~~ Insertion Sort ~~~~~

In *Insertion Sort*, we iterate for k from 1 up to $n-1$. We keep the first k elements — $a[0], a[1], \dots, a[k-1]$ — sorted, and we insert $a[k]$ among them where it belongs:

```
/**
 * Sorts n values in array a in ascending order.
 * Preconditions: array a contains a[0], ..., a[n-1]; n >= 1
 */
public void insertionSort(int[] a, int n)
{
    for (int k = 1; k < n; k++)
    {
        int temp = a[k];
        int i = k;
        while (i > 0 && a[i-1] > temp)
        {
            a[i] = a[i-1];
            i--;
        }
        a[i] = temp;
    }
}
```

In this version of Insertion Sort, if the array is already sorted, then the inner loop runs just one comparison and we immediately break out of it. Then the method needs a total of $n-1$ comparisons. This is the best case: instead of *quadratic* time, the method executes in *linear* time.

The worst case for this implementation of Insertion Sort is when the array is sorted in reverse order. Then the inner loop runs $k-1$ times and the whole method will need as many comparisons as Selection Sort:

$$1 + 2 + \dots + (n-1) = \frac{n(n-1)}{2}$$

The average case is about half that number — still approximately proportional to n^2 .

The methods above are just examples of how Selection Sort and Insertion Sort can be implemented. Other variations are possible.

34

Consider the task of sorting the elements of an array in ascending order. Which of the following statements is true?

- (A) Selection Sort always requires more comparisons than Insertion Sort.
- (B) Insertion Sort always requires more moves than Selection Sort.
- (C) On average, Insertion Sort requires more moves than Selection Sort.
- (D) If the array is nearly sorted, with only a few pairs of adjacent elements swapped, Selection Sort will run faster than Insertion Sort.

 This question gives us a chance to review the properties of these two quadratic sorts. As we have seen, the statement in Choice A is false: although on average Selection Sort requires more comparisons, in the worst case (an array sorted in reverse order) Insertion Sort will take as many comparisons as Selection Sort. The statement in Choice B is false, too: in the best case, when the array is already sorted, Insertion Sort does not require any moves at all. (Selection Sort, with a slight modification, can also avoid making any moves when the array is already sorted.) The statement in Choice D is false: the opposite is true. The statement in Choice C is the vague part: what do we mean by “on average”? First, our array must be large enough to support some conclusions. Sorting an array of three elements will not be representative. Let’s assume that we set up an experiment where we generate a fairly large array of random numbers, sort it using each of the two algorithms, and count the number of moves required for each. Intuition tells us that on average Insertion Sort needs more moves. Indeed, the k -th iteration through the outer loop may require anywhere from 0 to k moves — $k/2$ moves on average. In Selection Sort, each iteration through the outer loop requires one swap, which can be counted as three moves. The answer is C. 

4.4 Recursion

You may find recursion pleasant or difficult, depending on your taste. Even if you happen to dislike it, you can still take a stab at the multiple-choice questions on recursion.

35

Consider the following method.

```
public void mystery(int n)
{
    if (n <= 0)
        return;

    for (int i = 0; i < n; i++)
    {
        System.out.print("-");
    }

    for (int i = 0; i < n; i++)
    {
        System.out.print("+");
    }

    System.out.println();
    mystery(n-1);
}
```

What is the output when `mystery(4)` is called?

(A) -----++++

(B) -----+
-----++
-----+++
-----++++

(C) -+
--++
---+++
----++++

(D) -----++++
-----+++
---++
-+

 This method calls itself — that's what recursion is. Notice two things about it. First, if $n \leq 0$, the method doesn't do anything. An exit from a recursive method, perhaps after some work but without recursive calls, is called the *base case* (or the *stopping case*). In this method the base case does nothing. Second, when the method calls itself, it does so with a parameter whose value is one less than the original. The parameter of a recursive call must be different from the parameter of the original call; it's usually decreased in some way in the direction of the base case, if the recursion is to terminate at some point.

Instead of trying to unwrap and trace all the recursive calls in this method, first try to reason more formally about its properties. The method prints some minuses followed by the same number of pluses. When called with $n = 4$, the method immediately prints one line with 4 minuses and 4 pluses. But that's not all: after printing the first line, the method calls `mystery(3)`, which must do the same thing as `mystery(4)` but on a smaller scale. The answer is D. 

Now suppose we change the `mystery` method in the previous question, placing the recursive call above the `for` loops. That results in the following question.

36

Consider the following method.

```
public void mystery(int n)
{
    if (n <= 0)
        return;

    mystery(n-1);

    for (int i = 0; i < n; i++)
    {
        System.out.print("-");
    }

    for (int i = 0; i < n; i++)
    {
        System.out.print("+");
    }

    System.out.println();
}
```

What is the output when `mystery(4)` is called?

< same answer choices as in Question 35 >

➦ This question is a bit trickier, but C and D are still the only plausible answers:

(C)	(D)
--+	-----+++
---++	-----+++
-----+++	---++
-----+++	--+

We have to choose C because the last thing `mystery(4)` does is print `-----+++`.

(If you are more mathematically inclined, you can reason as follows. `mystery(4)` prints a triangle pointing either up or down. Let's take a guess at this method's general behavior: say, "`mystery(n)` prints a triangle with n rows that points up." Suppose it's true for $n = 3$. Then `mystery(4)` first prints a triangle with 3 rows in the recursive call, then adds the longest fourth row. Our guess fits, so the answer should be C.) ➦

37

Consider the following method.

```
public void mysteryMix(String str)
{
    int len = str.length();
    if (len >= 3)
    {
        mysteryMix(str.substring(0, len / 3));
        System.out.print(str.substring(len / 3, 2*len / 3));
        mysteryMix(str.substring(2*len / 3));
    }
}
```

What is the output when `mysteryMix("la-la-la!")` is called?

- (A) ala-a
- (B) lla-l
- (C) la-la-la!
- (D) ala-la-la-l

 Many AP exam questions mix unrelated topics. This tough question tests both recursion and strings. We start with a string of nine characters, but immediately call `mysteryMix` recursively for a string of three characters. So it makes sense to first see what happens when we call, say, `mysteryMix("xyz")`. This call just prints the middle character, "y", and does nothing else: when `len` is 3 the two recursive calls do nothing. Now back to the original string of nine characters. The two recursive calls print one character each and `System.out.print` prints three characters, so the output must have five characters. This eliminates Choices C and D. The first character printed is the middle character in the first one-third of the string, which is "a". The answer is A. 

~~~~~ Recursive implementation of Binary Search ~~~~~

The description of the Binary Search algorithm is recursive in nature, and the algorithm can be implemented recursively with ease.

38

Consider the following incomplete recursive implementation of Binary Search.

```
/**
 * Returns the position of the element equal to target, or -1
 * if target is not among the values a[left], ..., a[right].
 * Precondition: array a contains values stored from a[left]
 *                  to a[right], sorted in ascending order
 */
public int binarySearch(int[] a, int left, int right, int target)
{
    int targetPos = -1;

    < statement 1 >
    {
        int middle = (left + right) / 2;
        if (target == a[middle])
            < statement 2 >
        else if (target < a[middle])
            targetPos = binarySearch(a, left, middle - 1, target);
        else
            targetPos = binarySearch(a, middle + 1, right, target);
    }

    return targetPos;
}
```

Which of the following could be used to replace `< statement 1 >` and `< statement 2 >` so that the `binarySearch` method works as intended?

<statement 1>

- (A) while (left <= right)
 (B) while (left < right)
 (C) if (left <= right)
 (D) if (left < right)

<statement 2>

return targetPos;
 targetPos = middle;
 targetPos = middle;
 return middle;

👉 In the choice between while and if, if wins, because this is a recursive solution and recursion replaces iterations. This eliminates Choices A and B. In Choices C and D, either one works for Statement 2, but Statement 1 in Choice D misses the case in which `left == right`. The answer is C. 📌

4.5 Mergesort

Mergesort is a recursive sorting algorithm based on the “divide and conquer” principle. It takes, on average, $n \log n$ comparisons, as opposed to order of n^2 comparisons in quadratic sorts. This difference can be very significant for large arrays. For example, for 1024 elements, Mergesort may run 100 times faster than Selection Sort.

The idea of Mergesort is simple: divide the array into two approximately equal halves, sort (recursively) each half, then merge the two sorted halves into one sorted array (Figure 4-2). Mergesort usually requires a temporary array for holding the two sorted halves before they are merged back into the original space.

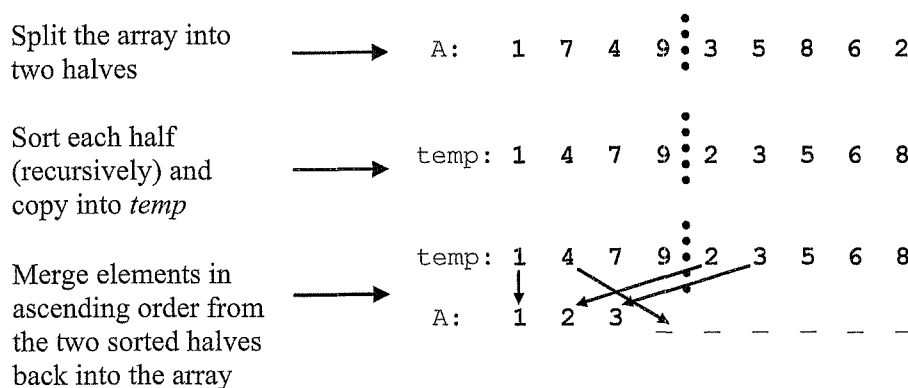


Figure 4-2. Mergesort

39

Consider the following implementation of Mergesort.

```
/**
 * Sorts values a[n1], ..., a[n2] in ascending order.
 * Precondition: 0 <= n1 <= n2 < a.length
 */
public void sort(int[] a, int n1, int n2)
{
    if (n1 == n2)
        return;

    int m = (n1 + n2) / 2;

    sort(a, n1, m);
    sort(a, m+1, n2);

    if (a[m] > a[m+1])        // Optional line
        merge(a, n1, m, n2);
}
```

Compare it with a more conventional version that doesn't include the "optional line" (the extra `if` statement). Suppose `a` has 8 elements and `sort(a, 0, 7)` is called. For which of the following values in `a` will the version with the extra `if` statement work faster than the version without it?

- (A) 1 2 3 4 5 6 7 8
- (B) 5 6 7 8 2 1 4 3
- (C) 2 1 4 3 6 5 8 7
- (D) All of the above

 A typical implementation of Mergesort doesn't skip the work even when the array is already sorted. The slight change proposed in this question allows Mergesort to skip all the merging and quickly establish that an array is already sorted, as in Choice A. This version also avoids merging when the array is partially sorted, namely when all the values in the left half of the array are smaller than any value in the right half, as in Choice C. In that case, after the two recursive calls to `sort`, the array becomes sorted and the call to `merge` is skipped.

Since the algorithm is recursive, it will also save time when some portions of the array are sorted or partially sorted, even if the whole array isn't. In Choice B, for example, the left half is sorted and the right half is partially sorted. The answer is D. 

